

Training a coding agent through a harness you did not write

Sergio Paniego Blanco ([@sergiopaniego](#))
Machine Learning Engineer @ Hugging Face

Where are the labs training their models?



In the last year, we all moved to working through coding agents (harness + model)

- Claude Code, OpenCode, Codex, Pi...

The labs train their models **on those same harnesses (even on several)**

Can we replicate that openly?

<https://huggingface.co/blog/sergiopaniego/trl-openenv-harness-training>

<https://huggingface.co/blog/agent-glossary>

<https://huggingface.co/blog/sergiopaniego/rl-environments-2026>

<https://huggingface.co/blog/sergiopaniego/rl-environments-story>

Agent = model + harness

The harness **calls the model, handles tool calls,** and **decides when to stop**

We can train a model on them (RL envs)

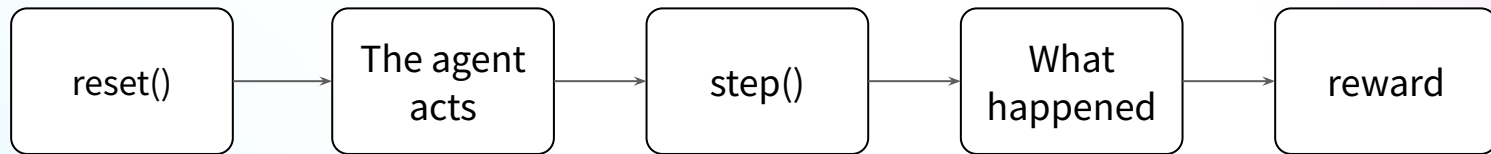
<https://huggingface.co/blog/agent-glossary>

<https://huggingface.co/blog/sergiopaniego/rl-environments-story>

A harness is an RL env

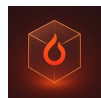
An env is where the model acts and gets consequences

- State: filesystem, repo, running process
- Actions: write (file), bash (execute), edit (file)
- Consequences: tests pass?



TRL, OpenEnv and more

 → the trainer ([AsyncGRPO](#))



[OpenEnv](#) → RL envs (harnesses)

And everything stays inside HF:

- Model: [hf.co/Qwen/Qwen3-8B](https://huggingface.co/Qwen/Qwen3-8B)
- Dataset: [hf.co/datasets/agentica-org/DeepCoder-Preview-Dataset](https://huggingface.co/datasets/agentica-org/DeepCoder-Preview-Dataset)
- [Trackio](#)
- [hf jobs](#)
- [hf sandboxes](#)

Why this is awkward to train

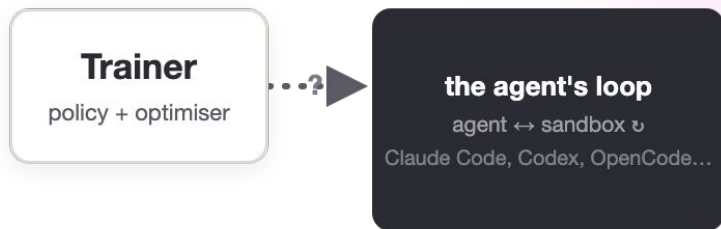
GRPO (RL) assumes the trainer drives every step: it prompts, samples, stops, scores (white box)

When training using a harness, it does all of that directly (black box)

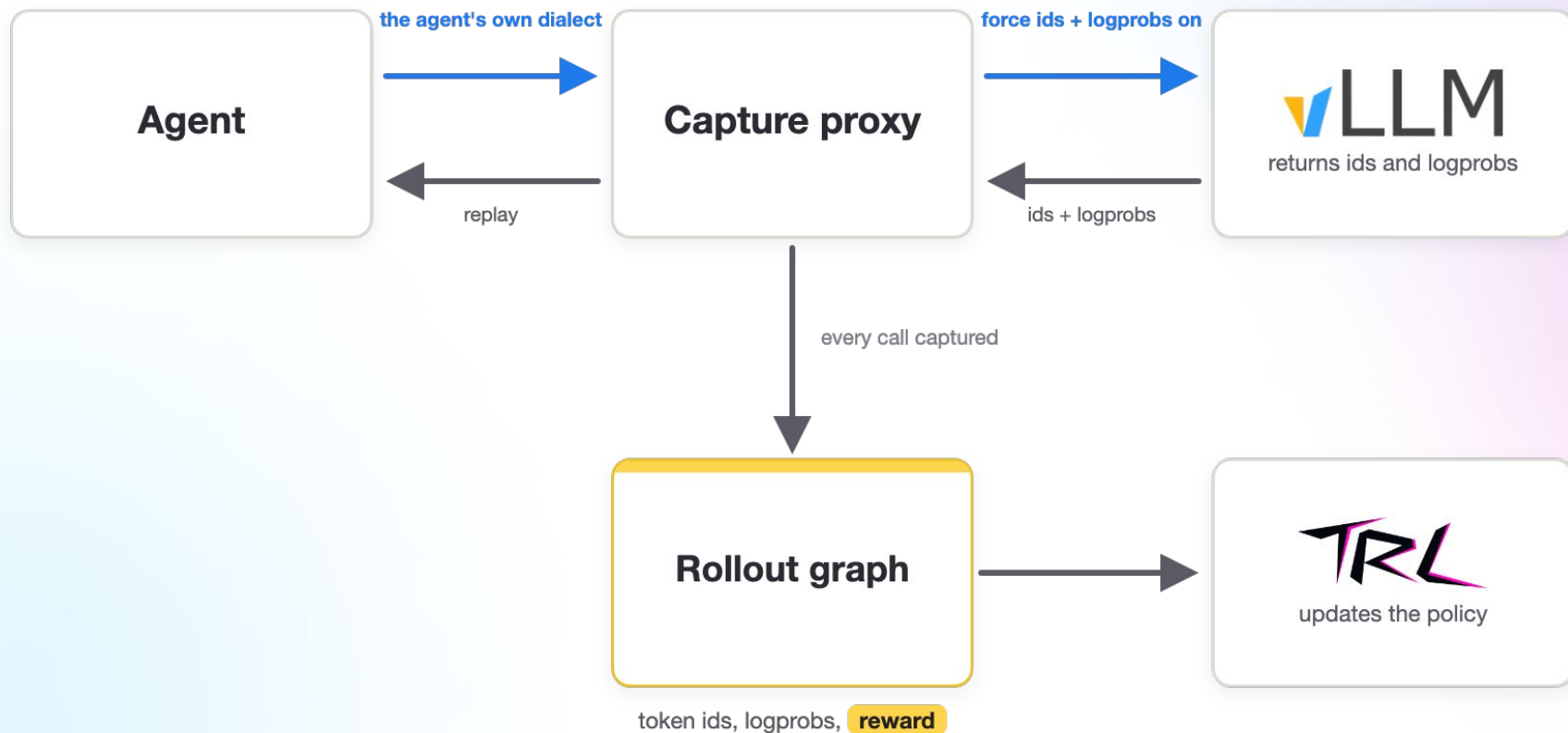
White box



Black box



Capturing what the agent does



In TRL

```
from opencode_env.harness import OpenCodeSessionFactory
from opencode_env.sandbox import HFSandboxBackend

factory = OpenCodeSessionFactory(
    config=config,
    sandbox_backend=HFSandboxBackend(image=image, flavor=flavor), # each rollout gets its own remote sandbox
    mode="transparent_proxy", # captures token ids + logprobs on the wire
    verifier=DeepCoderStdinVerifier(tests_by_id), # runs the hidden tests -> reward
)
```

```
from trl.experimental.async_grpo import AsyncGRPOTrainer
from trl.experimental.async_grpo.openenv_harness import HarnessRolloutWorker, has_tool_call

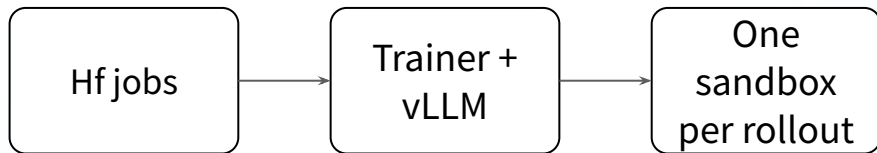
worker = HarnessRolloutWorker(
    harness_session_factory=factory, # boots agent + sandbox + proxy per rollout
    harness_adapter=None, # loop-owning: the agent runs its own loop
    rollout_reward_fn=opencode_reward, # dense verifier + degeneracy penalties
    train_turn_fn=has_tool_call, # reinforce action turns, not prose
    agent_turn_fn=opencode_agent_turns, # drop opencode's title and summarizer calls
)

trainer = AsyncGRPOTrainer(model="Qwen/Qwen3-8B", train_dataset=tasks, rollout_worker=worker)
trainer.train()
```

https://github.com/huggingface/trl/blob/main/examples/async_grpo_opencode/opencode_hf_sandbox.py

One sandbox per rollout

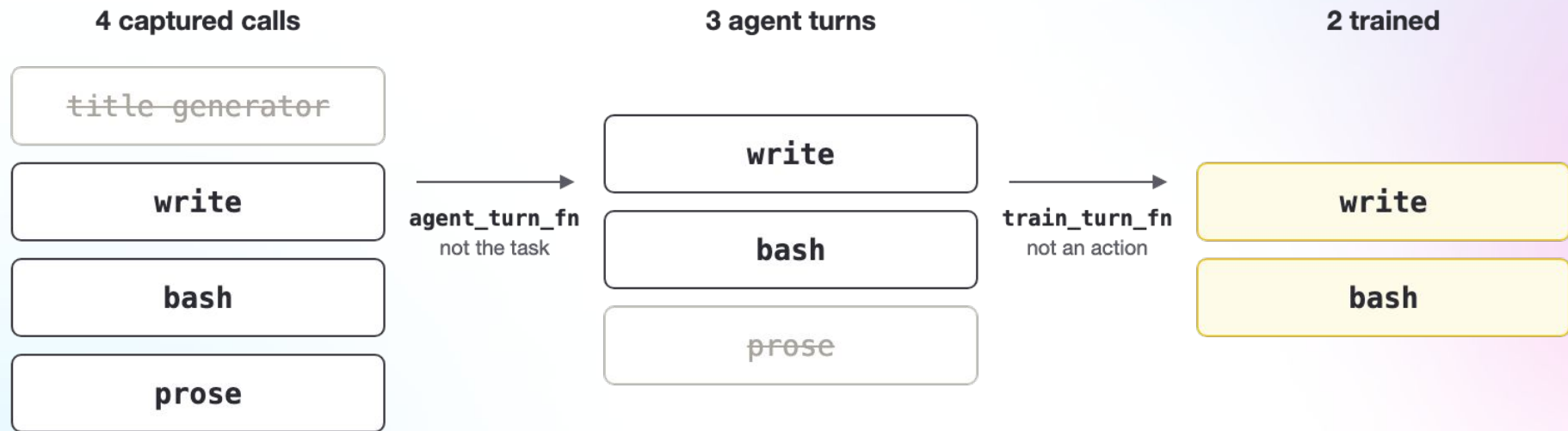
Everything runs in HF jobs and inside HF sandboxes



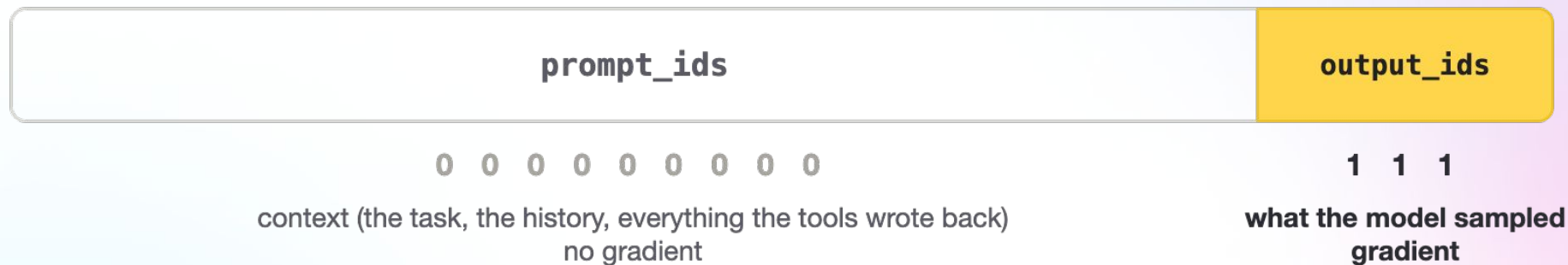
One sandbox is created per rollout → inside it, it runs a pre-baked isolated untouched dockerized opencode [image](#) for the particular task and model

<https://huggingface.co/settings/jobs>

What the proxy captured



Inside a trained turn



If you re-tokenize the conversation, you might train on something else. So the proxy records the exact ids the engine produced with their logprobs.

Where the reward comes from

Test types:

- Visible: examples in statement. Agent sees them → we ask it to run them
- Held-out: tests it never sees (frac below)

```
def opencode_reward(outcome) -> float | None:
    bash = outcome.tool_calls_by_name.get("bash", 0)
    if frac is None: return None          # unscorable, dropped from the group baseline
    if bash == 0:    return -0.1          # never ran its code
    base = 0.0 if outcome.timed_out else frac # dense: fraction of held-out tests passed
    over = max(0, outcome.tool_call_count - 30)
    return base - min(0.5, 0.03 * over)    # bounds runaway edit/bash loops
```

Reviewing a quick training run

- Qwen3-8B in 10 steps
- 2xh200, one hf job (1 GPU trainer, 1 GPU vLLM). One remote sandbox per rollout (no GPU)

<https://huggingface.co/spaces/sergiopaniego/opencode-hf-sandbox>

Multi-harness training

- OpenEnv now supports Harbor's harnesses which brings a lot of new RL envs ([PR #1036](#))
- Opens the door for multi-harness training via TRL ([PR #6948](#))



Thanks!

Sergio Paniego Blanco (@[sergiopaniego](https://twitter.com/sergiopaniego))
Machine Learning Engineer @ Hugging Face